Improving LLM-assisted code generation through the use of architectural documents and implementation plans.

Humberto Cervantes
Universidad Autónoma Metropolitana
Iztapalapa
CDMX, Mexico
hcm@xanum.uam.mx

Rick Kazman
University of Hawaii
Honolulu, USA
kazman@hawaii.edu

Yuanfang Cai
Drexel University
Philadelphia, USA
yuanfang.cai@drexel.edu

Abstract

In this paper, we investigate LLM-assisted code generation using architectural documentation as a first-class input. Building on prior work that guides LLMs through Attribute-Driven Design (ADD), we propose a workflow in which the LLM synthesizes an implementation plan from requirements and an architecture document, and then generates code with human oversight. Using a ReAct-style agent as a case study, we perform three experiments and evaluate architectural conformance and functional correctness of a primary use case. In addition we capture size and modularity metrics using the DV8 tool. Our results show that incorporating architectural documentation substantially improves conformance and that adding an explicit implementation plan further enhances functional correctness and modularity.

CCS Concepts

• **Software and its engineering → Automatic programming**.

Keywords

Large Language Model, AI assisted software development, code generation

ACM Reference Format:
Humberto Cervantes, Rick Kazman, and Yuanfang Cai. 2026. Improving LLM-assisted code generation through the use of architectural documents and implementation plans.. In *3rd International Workshop on Designing Software (Designing '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3786152.3788588

1 Introduction

The use of LLMs in software development has exploded in recent times. Many LLM-assisted approaches generate source code from natural language descriptions (e.g., prompts, function comments, or extensive requirement specifications), however, there are few approaches that generate code from design artifacts such as architecture documentation.

In [2], we explored the use of LLMs to assist in the design of software architectures and we observed that good results can be obtained by providing the LLM with guidance on the design process. More specifically, we provided the LLM with a description of the Attribute-Driven Design (ADD) process [1], a template of the architecture document and the requirements and we request through prompting the creation of a design plan. Once this plan has been created, the LLM interacts with the software architect to perform the design process. This approach results in a detailed architectural design document which provides detailed information to proceed to implementation.

Motivated by prior work showing that explicit design guidance can improve LLM-assisted architectural reasoning, we investigate whether similar benefits carry over to the code generation stage. Specifically, we design experiments that vary the form of design input provided to the LLM, including no plan, requirements only, and requirements plus an architecture document, representing increasing levels of explicit design guidance. Our objective is to examine whether introducing an explicit architecture document makes a difference in LLM-assisted code generation, and to what extent it influences the translation from design intent to implementation, compared to relying solely on requirements or ad hoc prompting.

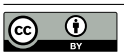
In this paper, we report the results of these experiments. Similar to what was previously discussed, we ask the LLM to plan the implementation based on the outputs from the design process and we proceed in a step-by-step approach where human and LLM collaborate. Our hypothesis is that the use of a documented design and a plan-based approach results in higher quality implementations than the direct generation of code from requirement descriptions.

From our hypothesis, we derive the following research questions:

- **RQ1:** Does the use of an implementation plan and an architecture document improve the architectural conformance of the generated system?
- **RQ2:** Does the code generated using our approach achieve higher functional correctness than code generated directly from requirements?
- **RQ3:** Does the code generated using our approach achieve higher modularity and maintainability metrics than code generated directly from requirements?

Our main contributions include a novel approach to code generation based on a plan-based approach and the use of architectural design documents. To evaluate our approach, we conducted three case studies of implementing a ReAct-style agent, in which an LLM generates the source code (1) from both the architecture document and implementation plan, (2) from the architecture document only, and (3) directly from the requirements. The results indicate that incorporating architectural documentation substantially improves conformance, while adding an explicit implementation plan further enhances functional correctness and modularity.

The structure of the paper is as follows: section 2 presents related work, section 3 discusses our approach, section 4 presents our



This work is licensed under a Creative Commons Attribution 4.0 International License.
Designing '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2385-8/26/04
https://doi.org/10.1145/3786152.3788588

experimentation followed by section 5 which analyzes our results. Section 6 discusses threats to validity and finally, section 7 presents a conclusion and future work.

2 Related Work

In the survey from Jiang et al. [3], the authors provide an extensive review of literature focused on the ability of Code LLMs to generate source code from natural language descriptions. However, the survey does not mention any specific approaches where code is generated from design artifacts (such as architectural specifications, UML, or other formal design documents). Another more recent survey from Dong et al. [8] also does not describe specific approaches of code generation from design artifacts and, furthermore, the authors mention that code generation agents have limited capabilities in overall software architecture understanding, which limits their use in large projects. They also mention that current LLM-based agents cannot fully understand and utilize multimodal information, including architectural sketches.

The work from Kazai et al [5] evaluates the capabilities of a Large Language Model (LLM) to perform model transformations. Their experiment focused on translating Unified Modeling Language (UML) class diagrams encoded in XML Metadata Interchange (XMI) format to object-oriented code (Java). The authors designed a semi-automatic pipeline to evaluate ChatGPT's performance in performing these transformations. Their results indicate that LLMs like ChatGPT are currently (August 2024) effective in handling simple to intermediate model transformations, but their performance declines substantially when the complexity of the models increases

In [6], Sadik et al. demonstrate the use of GPT-4 to generate deployable code from UML models in an Agile MDD context. The approach converts UML diagrams (augmented with OCL constraints and domain ontologies) into text and feeds them to an LLM. Evaluations show the auto-generated Java/Python code aligns with expected behaviors and manageable complexity, suggesting LLMs can serve as universal code generators for models.

While some studies have recently appeared exploring the abilities of LLMs to make architecture decisions, such as [9] and [10], we are only aware of one study [11] that has explicitly examined the abilities of LLMs to generate code from architectural descriptions, and this study only explores the rather limited domain of serverless functions. This paper is an empirical study that evaluates whether LLMs can generate entire software architecture components. The study focuses on serverless (FaaS) functions as manageable size components. It uses 5 state-of-the-art code LLMs with prompts providing varying system context to regenerate masked functions in open-source projects. The authors found that LLMs can often produce working code for a missing module when given a description and system context, though the approach worked best for small, encapsulated components.

There appear to be few research proposals that focus on generation of code from design artifacts. Interestingly, there are also few approaches focused on generating designs using LLMs [2]. In the related work that we found, none of the approaches is similar to what we propose. However, just prior to submitting this paper, we were informed about the existence of an agentic development environment (ADE) named Kiro [1] which was recently released and is still in preview mode. This ADE follows a similar approach to what we propose in this paper, and we compare it to our approach in section 5.4.

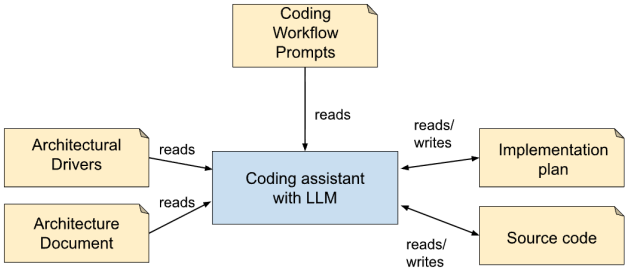


Figure 1: Elements of the approach

3 Approach

An overview of our approach is presented in Figure 1. The approach is based on the use of an LLM-based coding agent (Cursor v1.6.45 with automatic model selection), as the code-generation process involves working simultaneously with multiple project files. Cursor is an AI-enhanced code editor that integrates LLMs into the development environment to support code generation, refactoring, and testing across multiple files via natural-language interaction. Our objective is not to evaluate Cursor as a standalone code generation tool or to assess the functional correctness of the generated code. Instead, we focus on how designers should collaborate with an LLM to generate code from design artifacts, and how different forms of design input generate code with different quality. Figure 1 shows the different artifacts that are used in the process. The left side presents inputs to the process while the right side presents the outputs, which are generated through prompts from a code-generation workflow. We will now detail these elements.

3.1 Inputs

3.1.1 Architectural Drivers. The requirement specification is structured according to the main categories of architectural drivers including:

(1) Functional requirements. This is functionality that is critical to achieve the business goals that motivate the development of the system. These are typically captured as user stories or use cases.
(2) Quality attributes. They are defined as being measurable or testable properties of a system that are used to indicate how well the system satisfies the needs of its stakeholders.
(3) Constraints. These are decisions over which the architect has little or no control, they can be technical or non-technical.

Architectural drivers are described in a markdown document that facilitates processing by an LLM and also facilitates display so that humans can easily read them.

[1] https://kiro.dev/

3.1.2 Architecture Document. The architecture document is the result of a number of ADD iterations performed following the process described in [2]. This document typically includes a structure similar to what follows:

```
1. Introduction
2. Context diagram
3. Architectural Drivers
4. Container Diagram
5. Component Diagrams
6. Sequence Diagrams
7. Deployment Diagram
8. Interfaces and Events
9. Design decisions
```

Similar to the requirements document, this is a markdown document but additionally, it includes diagrams that are described using the mermaid syntax, which is also well handled by LLMs. It should be noted that the design that is documented does not represent a Big Design Up Front of the system, it is the result of a number of ADD iterations that address some of the architectural drivers and it is refined and extended as the system evolves. An example that we will be using in this paper is accessible online [2].

3.2 Outputs

3.2.1 Implementation plan. Wang et al. [4] describe an approach called Plan-and-Solve Prompting where the LLM is instructed to Devise a plan to divide the entire task into smaller subtasks. Once the plan is created, the LLM is asked to Carry out the subtasks according to the plan. In our work, we follow a similar approach, where we ask the LLM to create an implementation plan. The prompt used to create the plan is the following:

```
Please review the documentation in the following
files @requirements.md @architectureDesign.md
I would like you to propose an implementation plan
for the design and requirements that you have
just read. Create an implmentationPlan.md document
where you describe how you will implement these
specifications. The plan should describe a number
of phases and each phase should include the main
objective, a number of activities and tasks and
deliverables associated to the phase.
```

An example of the (partial) output of this prompt is the following:

```
### Phase 1: Foundation Setup
**Objective**: Establish project foundation, basic
structure, and core infrastructure #### 2.1.1
Project Initialization
- [ ] Create Spring Boot 3.x project with Maven
- [ ] Configure project structure following Clean
Architecture
- [ ] Set up package organization (domain,
application, infrastructure, presentation)
- [ ] Configure Spring Boot dependencies and
auto-configuration
- [ ] Set up logging framework (Logback) with
structured logging
- [ ] Configure application properties for different
environments
#### 2.1.3 Core Domain Models
- [ ] Implement basic domain entities (Agent,
Conversation, Message, Tool)
- [ ] Create value objects (ReActState, ReActContext,
ReActResult)
- [ ] Define domain interfaces and contracts
- [ ] Implement basic validation rules
**Deliverables**:
- Working Spring Boot application
- Basic domain model structure
- Local development environment
- CI/CD pipeline foundation
```

3.2.2 Source code. Once the implementation plan has been created, we ask the LLM to follow the plan along its different phases.

```
Consider the @requirements.md for this system and
the @implementationPlan.md. Let's generate the code
for this application, we will proceed phase by phase
of the implementation plan, let's start with phase
1.
```

For this study, we asked the LLM to run the tests at the end of each phase to ensure that the code compiled and tested correctly before continuing the development. As we will discuss later, this is not the ideal approach as a human should review and suggest corrections before proceeding to obtain better results.

4 Experimentation

To answer our research questions, we perform a number of experiments using a case study focused on the development of an LLM agent that implements the ReAct (reason and act) pattern [12]. The requirements for the agent can be found online [3].

The process described in [2] was followed to design the architecture of the system, the architecture document can also be found online, see the footnote with the link in section 3.1.2. The experiments that we conduct are the following:

[2] https://github.com/otrebmuh/LLM-agent/blob/main/Documentation/architectureDesign.md

[3] https://github.com/otrebmuh/LLM-agent/blob/main/Documentation/requirements.md

- **Experiment 1:** Generate source code following our approach, evaluate the resulting implementation and study if the code is aligned to the design.
- **Experiment 2:** Generate source code only providing the LLM with the architecture document but not requesting an implementation plan to be created, evaluate the resulting implementation and observe if the code is aligned to the design. Compare with the results from the first experiment.
- **Experiment 3:** Generate source code directly from requirements, without any plan or design document. Compare implementation from code of Experiment 1 with respect to requirement satisfaction.

Experiments 1 and 2 helps us answer RQ1, experiment 3 helps us answer RQ2. To answer Q3, we measured the modularity and maintainability of the source code generated in each experiment use the DV8 tool [13–15].

4.1 Experiment 1: Code Generation With Full Approach

Name in Diagram	Experiment 1 Name in Code (if present)	Experiment 2 Name in Code (if present)
Container diagram (section 4)		
Agent Controller	`ConversationController`	`ConversationController`
Configuration Controller	`ConfigurationController`	`ConfigurationController`
Health Controller	`HealthController`	`HealthController`
Agent Service	`AgentService`	`AgentService`
Conversation Service	-	`ConversationService`
Configuration Service	-	`ConfigurationService`
Tool Registry Service	-	-
Agent	`Agent`	`Agent`
Conversation	`Conversation`	`Conversation`
Message	`Message`	`Message`
Tool	`Tool`	`Tool`
Configuration	-	-
Configuration Repository	-	-
Conversation Repository	`ConversationRepository`	`ConversationRepository`
LLM Provider Adapter	`LLMProviderPort` + `LLMProviderAdapter`	-
Tool Executor	`ToolExecutor` (interface) + `ToolExecutorImpl`	-
Monitoring Service	-	`MonitoringService`
Configuration Properties	Spring Boot `@ConfigurationProperties`	Spring Boot `@ConfigurationProperties`
Configuration Validator	-	-
Environment Config Provider	-	-
Configuration Refresh Service	-	-
Agent Domain Entity Component Diagram (section 5.1)		
ReAct Orchestrator	`ReActOrchestrator`	`ReActOrchestrator`
ReAct State Manager	`ReActStateManager`	`ReActStateManager`
Reasoning Engine	`ReasoningEngine`	`ReasoningEngine`
ThoughtProcess	`ThoughtProcess`	-
ReasoningStep	`ReasoningStep`	-
Acting Engine	`ActingEngine`	`ActingEngine`
Action Plan	`ActionPlan`	-
Action Step	`ActionStep`	-
Acting Engine	`ActingEngine`	Handles acting phase
ReActState	`ReActState`	`ReActState`
ReActContext	`ReActContext`	`ReActContext`
ReActResult	`ReActResult`	`ReActResult`
ActingEngine Tool Integration Component Diagram (section 5.5.1)		
Tool Selector	`ToolSelector` (interface) + `ToolSelectorImpl`	-
CapabilityBasedToolSelector	-	-
ToolContext	-	-
ToolExecutor	`ToolExecutor`	-
DefaultToolExecutor	`ToolExecutorImpl`	-
AsyncExecution	-	-
ToolErrorHandler	-	-
ToolResultProcessor	-	-

Table 1: Mapping between Container Diagram Elements and Code Implementation

For this experiment, we followed the approach described in section 3. The implementation plan is followed up to phase 4 included, as these phases cover the implementation of the code to support the functional requirements. The generated code can be found online (in `fullApproach` branch) [4]. Once the code has been generated, we perform several analyses to compare the source code with the architectural document.

The first analysis involves evaluating whether or not the elements from the container and component diagram are present in the code. To do this, we analyzed 3 component and one sequence diagrams in the architecture document[5] and assess if the elements shown in the diagrams are present in the code. Table 1 shows the results of this analysis for the 3 component diagrams. Together, these 3 diagrams include 41 elements and we observe that 27 of the elements established in the design are actually in the code (65 % in this sample). Some missing elements related to configuration are probably missing because implementing configuration and monitoring is phase 5 in the implementation plan, and this phase was not implemented (we only implemented up to phase 4). We observe, however, that some tasks in the plan were not translated to code. For instance, the plan mentions the creation of two LLM provider adapters for OpenAI and Ollama but the code only includes a default `LLMProviderAdapter` with a comment mentioning that it is a placeholder implementation.

A second analysis involves the study of a sequence diagram named "6.4 ReAct Execution Sequence (Iteration 2)", shown in Figure 2. This diagram supports the main use case where the ReAct pattern is executed. The diagram shows that the `AgentService` invokes a second service—`ReactExecutionService`—which then calls a `ReactOrchestrator`. In the code, the `AgentService` directly invokes the `ReactOrchestrator` which implements the loop shown in the diagram. The `executeReactCycle()` method in the diagram is present in the code, and the `reason()` method of the `ReasoningEngine` is also called in the code, as shown in the diagram. The diagram shows two methods that are being called in the `ActingEngine`, `planActions()` and `executeAction()`. In the code, we observe that the `ActingEngine` methods are indeed called, but that the names of the methods have changed slightly to `createActionPlan()` and `executeActionPlan()`, the actual steps of the execution plan being handled by the `ActingEngine`. Overall, the implementation of the sequence diagram is close to the proposed design; the only major difference being that there is no `ReactExecutionService` in the code. It should be noted that in this experiment we allowed the LLM to generate the implementation without giving any feedback.

4.2 Experiment 2: Code Generation Without an Implementation Plan

In our second experiment, we simply requested the coding assistant to create a full implementation of the code with the following prompt:

[4] https://github.com/otrebmuh/LLM-agent/tree/fullApproach
[5] https://github.com/otrebmuh/LLM-agent/blob/fullApproach/Documentation/architectureDesign.md

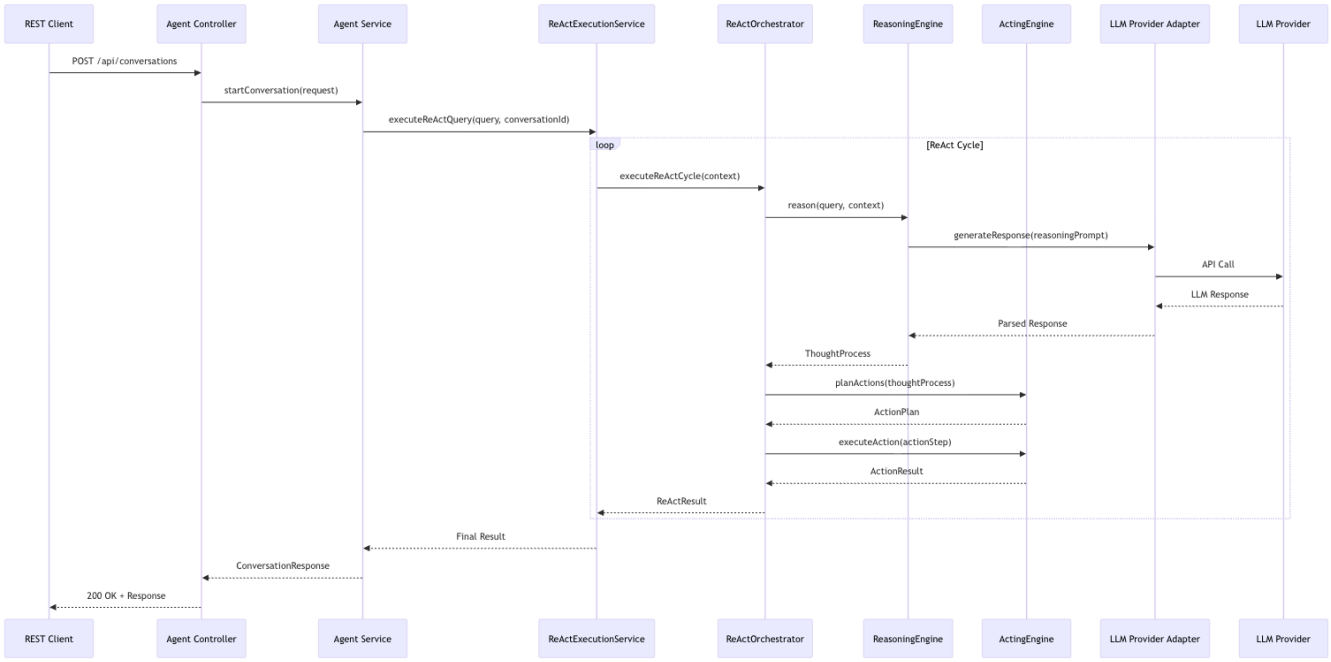


Figure 2: ReAct execution sequence diagram

```
Consider the @requirements.md and the
@architectureDesign.md. Please create a full
implementation of this design.
```

Once the implementation was created, we asked the LLM to run and correct the tests. It should be noted that while we do not ask the LLM to create and explicit implementation plan, Cursor now creates a basic plan of the work it is requested to perform and this cannot be disabled.

The plan it creates is simpler than the implementation plan used in the previous experiment and in this case it included 10 to-dos including:

- Creation of Maven Project Structure with Spring Boot 3.x and Java 17
- Implement core domain entities (Agent, Conversation, Message, Tool, Configuration)
- Implement ReAct pattern with ReasoningEngine, ActingEngine, and ReactOrchestrator
- Creation of tool intergration framework with basic tools (Calculator, FileOps, WebSearch)
- Implement application layer services (AgentService, ConversationService, ConfigurationService)
- Implement infrastructure layer (LLM Provider Adapter, Repositories, Monitoring)
- Create REST API controllers for conversations, configuration and health checks
- Set up Spring Boot configuration with validation and management
- Add health monitoring, metrics collection and observability features
- Create Docker configuration and deployment scripts.

The code for this second experiment can be found in the `noImplementationPlan` branch. While the overall implementation follows a similar structure to the one created in the first experiment, we observe a simpler implementation with less entities, with no ports and adapters. Table 1 shows that less elements of the diagram are present in the sample we studied (21 out of 41 or 51 %).

Interestingly, the implementation of the sequence diagram in Figure 2 is also not functional in the code. The `AgentService` implements the `startConversation()` method, but this method does not call the `ReactOrchestrator`, it only saves the conversation. Another method in the `AgentService` invokes the Orchestrator, but the method is never called. This experiment reveals that the use of a detailed implementation plan results in an implementation that is more functionally correct.

4.3 Experiment 3: Direct Code Generation

For the third experiment, we use the following prompt:

```
Review the @requirements.md and create a complete
implementation of these requirements.
```

As in the previous experiment, Cursor generates a high-level plan of tasks to be done which include:

- Analyze the requirements document to understand the complete system architecture
- Create the Spring Boot project structure with proper package organization
- Implement the ReAct pattern core agent functionality

- Create REST API endpoints for agent interaction
- Implement multi-LLM provider support (OpenAI, Anthropic, Ollama)
- Create flexible tool integration framework
- Implement memory management capabilities
- Implement token consumption monitoring and limiting
- Implement authentication, authorization, and security measures
- Implement health monitoring and observability
- Create configuration management and application properties

The result of the execution of the plan can be found in the `noPlanNoArchitectureDocument` branch of the repository.

This time, we observe a different implementation from the two previous experiments. The architecture of the solution is not structured using the Clean Architecture approach and, instead, it it structured using a layered approach. The implementation is simpler and the ReAct pattern is not implemented.

This experiment reveals that the use of the architectural document as an imput to the implementation process makes a significant difference.

4.4 Modularity and Maintainability Metrics

While conformance checking is clearly important, we also aimed to evaluate whether and how the modularity and maintainability of the source code generated in each experiment differ (RQ3). In addition to the basic measures of file counts and LOC, we also employed *M-Score* [16], a recent modularity metric that assesses the extent to which a system is decoupled into independent modules, and *Propagation Cost (PC)* [17], a more well-established metric that measures how tightly coupled a system is. M-Score and PC complement each other in most systems. That is, in general, the lower the PC, the higher the M-Score, and the better the system is modularized.

We use the DV8 tool [13] to calculate these metrics for the three code bases as well as recording their file counts and lines of code (see Table 2). To ensure accurate computation of M-Score and PC, we generated a design structure matrix (DSM) [14, 15, 17] for each code base to model the various dependencies among the code entities. After that, we removed all the auto-generated test files and their associated dependencies. We then used DV8 to extract the corresponding M-Score and PC values for these non-test-related files.

Table 2 presents the metrics for the three experiments. The DV8 analysis aligns with our qualitative observations: the code base from Experiment 1 contains the largest number of files and lines of code (LOC), followed by Experiment 2, while Experiment 3 produces the fewest files and LOCs. As discussed earlier, the code base generated in Experiment 3, which lacks both an implementation plan and an architecture document, is largely incomplete and does not conform to a clean architectural structure.

Interestingly, the modularity measures also reveal that the code generated directly from the requirements (Ex 3) has the highest PC score, indicating that its files are more tightly coupled than those in the other two code bases. Conversely, the source code generated using the full approach (Experiment 1) exhibits the highest M-Score, and its PC score is comparable to that of Experiment 2. These results provide early evidence suggesting that incorporating an implementation plan has the potential to significantly improve modularity, and hence maintainability.

Table 2: Comparison of Experimental Results

Experiment	Files	LOC	Non-test Files	Propagation Cost	M Score
Ex 1. Full Approach	67	4665	64	9.74%	86.73%
Ex 2. No Plan	50	3898	41	9.64%	69.67%
Ex 3. No Plan, no Architecture Document	37	2106	32	14.39%	75.77%

5 Results Analysis

Considering the results from our experiments, we now proceed to answer our three research questions.

5.1 Architectural Conformance

Our first question RQ1 is "*Does the use of an implementation plan and an architecture document improve the architectural conformance of the generated system*?". What we observe from the experiments 1 and 2 is that both implementations are relatively close to the design. However, the use of an architecture document as an input to the LLM seems to play a more important role than that of the implementation plan. While both implementations are aligned with the documented design, the code that was generated without the use of an implementation plan is less complete. We only analyzed a subset of the diagrams in the architecture document, however, the metrics from the analysis with the DV8 tool also point in this direction. The "full approach" experiment one has the largest number of files (67) and lines of code (4665) compared to the other experiments which have 50 files and 3898 LOC and 37 files and 2106 LOC, respectively.

5.2 Functional Correctness

Our second question RQ2 is "*Does code generated using our approach achieve higher functional correctness than code generated directly from requirements?*". From the analysis of the main use case for the system, which was presented in the sequence diagram shown in Figure 2, we observe that the use of an implementation plan appears to improve the odds of generation of code that is functionally correct, as judged by how well the primary functional requirement (FR-01 ReAct Pattern Implementation) is satisfied. However, when no architecture document is provided as an input to the LLM, the implementation is partial and not functionally correct. As we observed in the code review, the code generated without a plan and an architecture document did not even implement the ReAct loop.

5.3 Modularity and Maintainability

Our third question RQ3 is "*Does the code generated using our approach achieve higher modularity and maintainability metrics than code generated directly from requirements?*". We observe that the M-Score for the full approach is the highest. The higher M-score reveals that the implementation that was produced following our approach is more modular and easier to maintain. It seems like the use of an implementation plan was also a contributing factor here, as the implementation that was generated using no implementation plan but with an architecture document had significantly reduced the degree of coupling, that is, with a much better PC score.

5.4 Kiro Implementation

Kiro is an Agentic Development Environment (ADE), essentially an IDE that integrates an LLM that helps in code development and that can use tools to achieve this goal. This tool was released in July 2025 and was created by Amazon Web Services, building on top of the Claude Sonnet LLM. It allows developers to generate code by first creating a requirements document, a design document (spec) and a plan. Hence Kiro follows a similar approach to what we have presented in this paper. We provided Kiro with the same requirements used in the experiments previously described and allowed it to follow its proposed workflow. We accepted what the tool proposed until it ran through all the tasks of its implementation plan. The results can be found in the `kiroImplementation` branch of our repository. We observe that the default design document generated by Kiro [6] is much less detailed than the ones we use in our approach: it is short, and includes just two component diagrams and does not include any sequence diagrams (although they can be requested from the tool. The implementation plan is, however, quite detailed, and more extensive than the one used in our experiments. An important feature of this tool is that it incorporates mechanisms to ensure that the implementation plan is followed precisely, something that is more difficult to achieve with our approach. After the execution of the plan, Kiro produced a more extensive implementation than our full approach experiment (219 source files) but with a similar architecture (following the tenets of Clean Architecture). The larger size may be due to its rigor in following the implementation plan, which was fully executed, and because we did not execute all of the phases in the plan of our experimentation, only the ones focused in the generation of the core of the solution. Given the short size of the design document and the limited information it provides, we were not able to perform a detailed analysis as we did for our experiments that used the architecture document as an input. A more detailed comparison of Kiro's implementation in this and other case studies is part of our future work.

6 Threats to Validity

As with any research there are a number of internal and external threats to validity that need to be reflected upon.

First, and most obvious: we have only performed a small number of experiments, within just a single application domain. This poses a threat to external validity. Exploring more case studies in varied domains is part of our future work. And in these case studies we will evaluate the functional correctness of any generated solutions.

In addition, the case study presented here is relatively simple—another external threat. But we believe that this case study is still interesting and valid because it is unlikely that the AI would have seen many examples of this type of system in its training data. Clearly, we need to explore systems with more complex requirements. And we need to be more rigorous about establishing the correctness of any AI-generated solutions.

The use of Cursor, or other proprietary AI-assisted Integrated Development Environments presents a threat to validity as these tools make many decisions "behind the scenes", such as automatic LLM selection or context management, for which we have little control. As mentioned in the approach section, the use of such a tool is necessary as these tools are capable of handling complex multi-file projects.

Due to the usage of Cursor, we recognize that the experimental interventions were not cleanly isolated. In the "no plan" condition, Cursor still auto-generated a simplified plan, and in the "requirements-only" condition, Cursor also generates scaffolded tasks. As a result, the intended distinctions between conditions were partially blurred, introducing a potential threat to internal validity. To address this limitation, our future work includes developing tooling support that enables stricter control over planning and scaffolding behaviors, allowing for more cleanly isolated experimental conditions.

Finally, the approach that we followed in this paper involved a small minimal of human feedback and guidance during the implementation process. This approach was chosen to avoid introducing additional variables such as the level of experience of the reviewer, type of feedback, and so on. Also, we believe that humans frequently engage in this behavior, where users blindly accept what the LLMs are producing, so it is valuable to evaluate the product of such an approach. We believe this is not the ideal pattern of interaction in practice. At the end of each implementation phase, a human should be able to review and provide feedback to the LLM. This will likely result in a better implementation.

7 Conclusions and Future Work

In this paper we have explored the use of LLM-based technologies to generate code based on architectural input. The results were, we believe, quite encouraging. Our early research results suggest that the use of design documents as inputs to LLM-based code generation gives better, more complete results than a simpler Vibe coding approach. Perhaps this seems like an intuitively obvious hypothesis, but we feel that only a rigorous approach to testing this intuition will clarify the precise nature of, and best use of, design documentation in the service of creating a working system.

The use of AI in all walks of life, including software engineering, is rapidly evolving, with new tools constantly being introduced. Thus a rigorous testing and evaluation framework—which facilitates rotation between IDEs and LLMs and automates as much of the evaluation as possible (including functional correctness, code quality, design quality, and design conformance)—is needed and this is part of our future work.

Furthermore, our brief experience with Kiro validates our approach. Kiro's process mirrors many of the same steps that we took, and its outputs appear to be quite similar to the ones obtained with our approach. This is merely anecdotal evidence at this point, and more experimentation is clearly needed.

The results of this study have also highlighted a problem in how humans are currently using AI tools to generate code. It is curious that people seem to largely skip design when using AI tools to generate code, as observed in the related work. Our results suggest that we should ask the LLMs to help us do design, not just to translate requirements to code.

Our future work in this direction has several dimensions. As mentioned above, we need to explore more case studies in varied domains, with varied complexity and different amounts of user

[6] https://github.com/otrebmuh/LLM-agent/blob/kiroImplementation/.kiro/specs/agent-application/design.md

feedback, to better understand the strengths and weaknesses of the use of AI in system creation, and the role of an explicit design step in the quality of the outputs.

But there are other ways in which AI can potentially aid in this process. For example, we could ask an AI to analyze the conformance of a proposed implementation with respect to the design document, and then make corrections based on the results. We already did such an analysis and the preliminary results seems encouraging. Follow the Evaluator-optimizer pattern from Anthropic to improve results.

In addition we believe that the approach discussed here can be implemented with tools other than AI-assisted IDEs. One possibility is to develop our own tools which allow us to completely control how context is managed and which LLM is selected (we are already developing such tool) and another possibility is to use an agent-based approach. The agent-based approach poses interesting challenges, including the design of the solution and exploring whether there are other ways to exchange information between an architecture agent and a coding agent to yield better results.

Finally, we need to be more rigorous in our evaluation of the outputs of this process. In this first set of experiments we only considered the size and "completeness" of the outputs, as well as modularity metrics. But other measures of goodness should be considered, such as the degree to which the outputs of this process are maintainable and understandable, and how well they can achieve the system's quality attribute requirements, such as performance, security, availability, and testability.

References

[1] Cervantes, H., & Kazman, R. (2024). "Designing Software Architectures: A Practical Approach (2nd ed.)". Addison-Wesley Professional. ISBN: 978-0-13-810802-1.

[2] Cervantes, H., Kazman, R., & Cai, Y. (2025). "An LLM-assisted approach to designing software architectures using ADD." *arXiv preprint* arXiv:2506.22688. https://arxiv.org/abs/2506.22688

[3] Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2024). "A Survey on Large Language Models for Code Generation." *arXiv preprint* arXiv:2406.00515. https://arxiv.org/abs/2406.00515

[4] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models." *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2609–2634. Association for Computational Linguistics. DOI: 10.18653/v1/2023.acl-long.147.

[5] Kazai, G., & Osei, R. A. (2024). "MODEL TRANSFORMATIONS WITH LLMS." Master of Science in Engineering Thesis in Software Engineering, Mälardalen University, Västerås, Sweden.

[6] Sadik, A. R., Brulin, S., Olhofer, M., Ceravola, A., & Joublin, F. (2024). "LLM as a Code Generator in Agile Model-Driven Development." arXiv preprint arXiv:2410.18489.

[7] Arun, S., Tedla, M., & Vaidhyanathan, K. (2025). "LLMs for Generation of Architectural Components: An Exploratory Empirical Study in the Serverless World." arXiv preprint arXiv:2502.02539.

[8] Dong, Y., Jiang, X., Qian, J., Wang, T., Zhang, K., Jin, Z., & Li, G. (2025). "A Survey on Code Generation with LLM-based Agents." *arXiv preprint* arXiv:2508.00083. https://arxiv.org/abs/2508.00083

[9] R. Dhar, K. Vaidhyanathan, and V. Varma, "Can LLMs Generate Architectural Design Decisions? - An Exploratory Empirical Study," in *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, pp. 79–89. IEEE, 2024.

[10] J. Jahić and A. Sami, "State of Practice: LLMs in Software Engineering and Software Architecture," in *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, pp. 90–100. IEEE, 2024.

[11] S. Arun, M. Tedla and K. Vaidhyanathan, "LLMs for Generation of Architectural Components: An Exploratory Empirical Study in the Serverless World," 2025 IEEE 22nd International Conference on Software Architecture (ICSA), Odense, Denmark, pp. 25-36, IEEE, 2025.

[12] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models." *arXiv preprint* arXiv:2210.03629. https://arxiv.org/abs/2210.03629

[13] Y. Cai and R. Kazman, "DV8: Automated Architecture Analysis Tool Suites," in *Proceedings of the 2019 IEEE/ACM International Conference on Technical Debt (TechDebt)*, pp. 53–54, IEEE, 2019. doi: 10.1109/TechDebt.2019.00015.

[14] R. Mo, W. Snipes, Y. Cai, S. Ramaswamy, R. Kazman, and M. Naedele, "Experiences Applying Automated Architecture Analysis Tool Suites," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE 2018)*, pp. 779–789, ACM, Montpellier, France, 2018. doi: 10.1145/3238147.3240467.

[15] Y. Cai, L. He, J. Qian, Y. Kochinski, N. Zhang, C. Jaspan, and A. Bianco, "Understanding Architectural Complexity, Maintenance Burden, and Developer Sentiment—A Large-Scale Study," in *Proceedings of the 47th International Conference on Software Engineering (ICSE '25)*, 2025.

[16] E. Pisch, Y. Cai, R. Kazman, J. Lefever, and H. Fang, "M-Score: An Empirically Derived Software Modularity Metric," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24)*, pp. 382–392, ACM, Barcelona, Spain, 2024. doi: 10.1145/3674805.3686697.

[17] A. MacCormack, J. Rusnak, and C. Y. Baldwin, "Exploring the Structure of Complex Software Designs: An Empirical Study of Open Source and Proprietary Code," *Management Science*, vol. 52, no. 7, pp. 1015–1030, July 2006.